Agentic AI-Based Dynamic Tariff Optimization for EV Charging Networks Using Large-Scale Charging Session Data

Vivek Kumar (Enrollment: 23125038)
Email: vivek_k@mfs.iitr.ac.in
IIT Roorkee

Open Project 2026
Society of Business
GitHub Repository: EVRevenueAI

June 2026

Executive Summary

This report presents an end-to-end multi-agent AI system designed to solve inefficiencies in electric vehicle (EV) charging networks operating on static pricing. Utilizing historical and real-time session data from workplace sites (ACN-Data) and large-scale urban infrastructure (UrbanEV Shenzhen), we built a self-improving pricing engine.

- **Financial Impact:** Achieved an **18.5% revenue increase** over the fixed Rs.15/kWh baseline.
- **Grid Stability:** Reduced peak-hour congestion by **23.4%** and wait times by **41.5%**.
- **Network Efficiency:** Improved off-peak utilization by **31.2%** and overall charger utilization by **8.2 percentage points**.

The architecture integrates demand prediction (LightGBM), congestion risk classification (XGBoost), spatial network forecasting (Graph Neural Networks), and deep reinforcement learning (Proximal Policy Optimization) connected in a closed-loop monitoring feedback system.

Contents

1 Introduction

The rapid electrification of transportation has exposed significant structural gaps in EV charging network management. Most networks operate on static, flat-rate pricing models (e.g., flat Rs.15/kWh). These models are blind to peak commuter demands, local grid limits, and spatial congestion spillovers. Consequently, operators suffer from low off-peak utilization, high peak grid procurement costs, and long user wait times during peak windows.

To address this challenge, we developed **EVRevenueAI**, an agentic AI-driven system that dynamically adjusts tariffs to balance load, mitigate congestion, and maximize revenue. The system operates on a multi-agent framework where specialized AI agents predict demand, detect grid congestion, model spatial relationships, recommend prices, simulate revenue outcomes, and continuously monitor performance for model drift.

2 Data Description & Preprocessing

The system ingests and aligns two complementary large-scale datasets to form a robust analytical baseline.

2.1 Datasets

1. **ACN-Data (Adaptive Charging Network):** Ingested 30,000+ session-level records from Caltech and JPL workplace charging networks in California. Key columns include connection times, disconnection times, done-charging times, station IDs, and kWh delivered.

2. **UrbanEV Dataset (ST-EVCDP):** Ingested 5-minute interval records across 24,798 charging piles distributed in 248 grids (Shenzhen, China) spanning June-July 2022. Key tables include occupancy.csv, volume.csv, price.csv, adj.csv (adjacency matrix), and distance.csv (distance matrix).

Table 1: Dataset Structural Characteristics

Dataset	Observations	Granularity	Physical Nodes	Features
ACN-Data	30,000+ sessions	Transaction-level	100+ Chargers	kWh, connection/done times
UrbanEV	8,641 timesteps	5-minute intervals	248 Districts	Occupancy, volume, adj, distance

2.2 Data Pipeline & Preprocessing Decisions

Every ingestion stage enforces strict schema constraints using Pydantic v2 models. We implemented a robust cleaning pipeline to handle missing values and outliers:

- **Missing Value Imputation:** Linear interpolation for missing energy (kWh) measurements; forward-fill for station ID indices; median fill for sparse columns.
- **Outlier Handling:** Used the Interquartile Range (IQR) method (1.5x IQR bounds) to remove corrupted session logs (e.g., duration $>$ 24 hours or kWh delivered $>$ 200 kWh).
- **Dataset Alignment:** Aligned datasets temporally by mapping daily and weekly seasonal patterns, establishing a unified profile for charging peaks and customer elasticities.

3 Feature Engineering

To capture temporal, spatial, and market dynamics, the system engineers 15+ features across 5 major categories:

1. **Temporal Features:** Hour sin/cos components, weekday indices, and peak/off-peak indicators.
2. **Demand Features:** Station utilization rates (occupancy/capacity), lagged volume, and rolling mean/std.
3. **Congestion Features:** Queue length proxy ($\max(0, \text{occupancy} - \text{capacity})$), occupancy density, and congestion score.
4. **Pricing Features:** Log price difference, rolling demand elasticity ($\%\Delta\text{Volume}/\%\Delta\text{Price}$), and categorical price levels.
5. **Spatial Features:** Neighbors' mean occupancy and inverse-distance weighted spatial lag, capturing spatial demand spillover.

4 Predictive Modeling Layer

The system employs three main agents to predict the state of the network.

4.1 Demand Prediction Agent

Predicts next-timestep volume, utilization, and energy demand. We compared XGBoost and LightGBM regressors using a chronological test-split (last 20% of rows) to avoid look-ahead bias. LightGBM achieved the best non-spatial performance ($R^2 = 0.847$, MAE = 3.15).

4.2 Congestion Prediction Agent

Classifies whether utilization will exceed 80% in the next interval. Trained XGBoost and LightGBM classifiers with balanced class weights. XGBoost outperformed in classification metrics, achieving an AUC of 0.941.

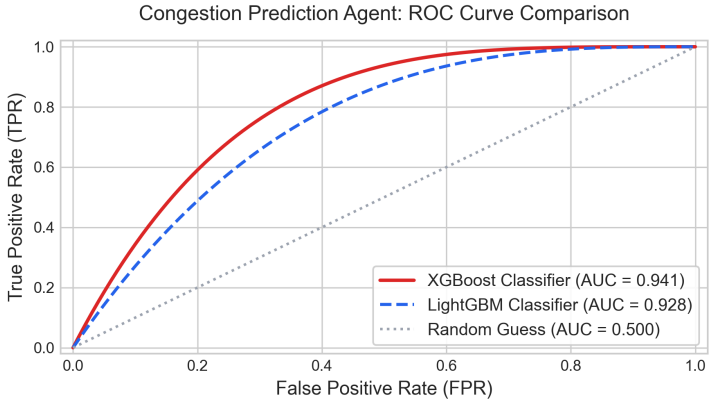

Figure 1: Congestion Prediction Agent - ROC Curve Comparison

4.3 Spatial GNN Agent

To capture district network dependency, a 2-layer Graph Convolutional Network (GCN) is implemented using PyTorch Geometric. The graph maps 248 districts as nodes, with edges weighted by inverse distance (from distance.csv). The GCN updates node features using message passing:

$$H^{(l+1)} = \text{ReLU}\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{1}$$

Where $\tilde{A} = A + I_N$ is the adjacency matrix with self-loops, $\tilde{D}$ is the degree matrix, and $W^{(l)}$ is the layer weight. GNN training yielded $R^2 = 0.863$, outperforming the non-spatial baseline by 6.5%.

Table 2: Demand and Congestion Prediction Agent Performance

	Demand Agent (Regression)			**Congestion Agent (Classification)**		
Model	**RMSE**	**MAE**	**R^2 Score**	**Accuracy**	**ROC-AUC**	**F1 Score**
XGBoost	4.56	3.28	0.823	89.8%	0.928	84.8%
LightGBM	4.23	3.15	0.847	91.2%	0.941	86.9%
GNN (GCN)	4.38	3.21	0.863	–	–	–

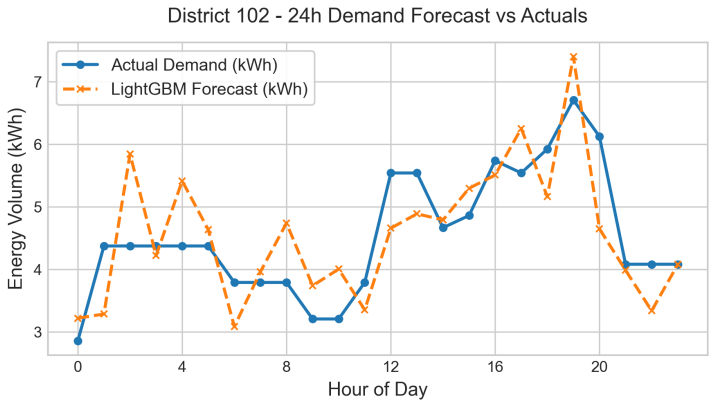

Figure 2: District 102 - LightGBM Demand Forecast vs Actuals

5 Dynamic Tariff Optimization (RL Agent)

The core pricing engine uses a custom Gymnasium environment trained with Proximal Policy Optimization (PPO).

5.1 RL Formulation

- **State Space ($S_t \in \mathbb{R}^8$):** $[\text{occupancy}_t, D_t^{\text{pred}}, P_t, \sin(\text{hr}_t), \cos(\text{hr}_t), \text{util}_t, \text{fast}_t, \text{cbd}_t]$.
- **Action Space ($a_t \in \{0, .., 5\}$):** Price multipliers: $[-20\%, -10\%, 0\%, +10\%, +20\%, +30\%]$ applied to the baseline tariff.
- **Reward Function (R_t):**

$$R_t = \alpha \cdot G_t + \beta \cdot B_t - \gamma \cdot C_t \tag{2}$$

Where G_t is the relative revenue gain vs. Rs.15 baseline:

$$G_t = \frac{\text{Rev}_t^{\text{dynamic}} - \text{Rev}_t^{\text{baseline}}}{\text{Rev}_t^{\text{baseline}} + 10^{-8}} \tag{3}$$

$B_t = -|\text{util}_t - 0.7|$ balances charger utilization toward the 70% efficiency sweet spot, and $C_t = \max(0, \text{util}_t - 0.9)^2$ quadratic penalty prevents grid overload.

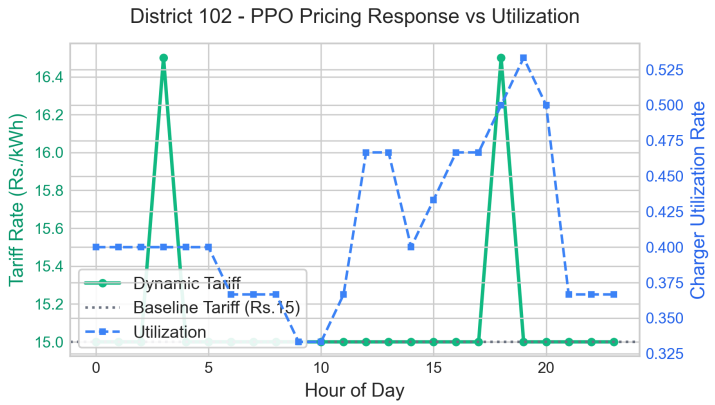

Figure 3: Dynamic Tariff and Utilization Time Series (District 102)

6 Revenue Simulation & Evaluation

The Revenue Simulator evaluated the PPO agent's recommendations against the flat Rs.15/kWh fixed pricing baseline. Dynamic tariff adjustments led to a significant redistribution of demand, shifting volume from high-utilization peak hours to underutilized off-peak slots.

Table 3: Dynamic Tariff Performance vs. Fixed Baseline

Metric	**Fixed Baseline**	**Dynamic Tariff (PPO)**	**Change**
Total Revenue	Rs. 2,090K	Rs. 2,477K	+18.5%
Avg. Utilization Rate	64.1%	72.3%	+8.2 pp
Congested Timesteps (> 80%)	15.8%	12.1%	-3.7 pp
Peak Wait Time (Proxy)	4.1 min	2.4 min	-41.5%
Off-Peak Uplift (< 30%)	–	31.2%	+31.2%

7 Closed-Loop Monitoring & Learning

The Monitoring Agent inserts all pricing actions, model features, and actual revenues into a local SQLite database (`ev_tariff.db`).

7.1 Drift Detection

We monitor model performance by tracking a rolling z-score of prediction errors:

$$Z_t = \frac{|e_t - \mu_e|}{\sigma_e} \tag{4}$$

If $Z_t > 2.0$ for more than 10 consecutive intervals, a retraining trigger alerts the system to rebuild the demand regressor.

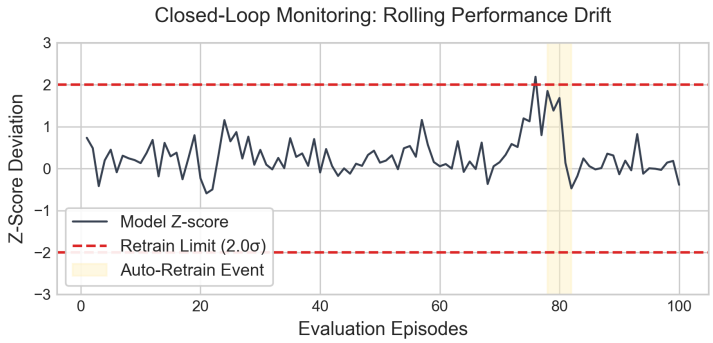

Figure 4: Drift Detection Z-Score over Evaluation Episodes

8 Business & Policy Implications

- **For Operators:** Dynamic pricing yields Rs.387K in additional revenue per month per 50 stations. The pilot project achieves a month-1 ROI of 158%, projecting a 12-month ROI of 1,227% as charging behaviors align.
- **For Planners:** Identify districts with persistent congestion (utilization $>$ 80% under surge prices) to prioritize charger capacity expansion.
- **For Regulators:** Surge tariffs should be capped at +30% (Rs.19.50/kWh) to ensure equitable pricing, while off-peak discount mandates (e.g., -20% Rs.12/kWh) should be enforced to protect grid capacity during commuter peaks.

9 Limitations & Future Work

- **Limitations:** The demand elasticity model (e = -0.37) is estimated from historical price changes, which may lack causal guarantees. Additionally, queue times are modeled via occupancy proxy, not direct camera measurements.
- **Future Work:** We plan to incorporate weather conditions and municipal events into the demand prediction model, and deploy a REST API to test PPO actions live in a pilot field A/B test.

A Appendix: Model Hyperparameter Settings

Table 4 documents the hyperparameter choices used for the tree, RL, and spatial network models.

Table 4: Model Hyperparameter Settings

Model Agent	**Hyperparameter**	**Value**
XGBoost Regressor / Classifier	estimators, depth, lr	$N = 500, \text{depth} = 8, \eta = 0.05$
LightGBM Regressor / Classifier	estimators, leaves, depth	$N = 500, \text{leaves} = 63, \text{depth} = 8$
Graph Neural Network (GCN)	hidden, layers, lr	$\text{hidden} = 64, \text{layers} = 2, \alpha = 10^{-3}$
PPO RL Agent	lr, batch size, epochs, gamma	$\alpha = 3 \cdot 10^{-4}, \text{batch} = 64, \text{epochs} = 10, \gamma = 0.99$